

UNIVERSITY OF BUEA



REPUBLIC OF CAMEROON
PEACE-WORK-FATHERLAND

Faculty of Engineering and Technology

Department of Computer Engineering

SOFTWARE REQUIREMENTS SPECIFICATION

*Design and Implementation of a Secured Mobile Telemedicine
Platform*

Document Version: 1.0

Prepared by: ABILATEZIE VIN-WILSON ANU

Matriculation Number: FE22A132

Supervisor: Prof. ELIE FUTE T.

University of Buea

Academic Year: 2025/2026

1. INTRODUCTION

1.1 Purpose

This Software Requirements Specification (SRS) document defines the complete functional requirements, non-functional requirements, system interfaces, constraints, and design boundaries for the Secured Mobile Telemedicine Platform developed as the final year project of ABILATEZIE VIN-WILSON ANU (FE22A132), Department of Computer Engineering, Faculty of Engineering and Technology, University of Buea, academic year 2024/2025.

The purpose of this document is to provide a comprehensive, unambiguous, and verifiable specification of all requirements that the proposed system must satisfy. This SRS serves as the binding technical reference for the design and implementation activities documented in Chapter Three and Chapter Four of the dissertation, and as the evaluation baseline against which the implemented system is assessed in the testing and evaluation activities of Chapter Four.

1.2 Scope

The system specified in this document is a cross-platform mobile application developed using the Flutter framework, targeting Android as the primary deployment platform with secondary compatibility for iOS. The system is designed to facilitate secure remote healthcare consultation between patients and verified healthcare professionals in Cameroon, with the following principal capabilities:

- Secure user registration, multi-factor authentication, and biometric login managed through Firebase Authentication.
- Healthcare professional credential verification through an administrative review and approval workflow.
- Appointment scheduling, management, and notification between patients and verified doctors.
- Real-time video and audio consultation through the Jitsi Meet SDK with adaptive bitrate control for low-bandwidth environments.

- Secure electronic medical records creation, storage, and patient-access management.
- Prescription document upload and delivery via Cloudinary secure media storage.
- Push notification delivery via Firebase Cloud Messaging.
- Bilingual English and French user interface implemented via Flutter's i18n framework.
- Comprehensive activity logging of all significant system interactions.

The system explicitly excludes: AI-based diagnostic support; pharmacy or medication delivery integration; health insurance or billing integration; full hospital information system integration; offline consultation capabilities; desktop or web browser deployment; and wearable device integration. These exclusions are documented in detail in Section 1.9 of Chapter One of the dissertation.

1.3 Definitions, Acronyms, and Abbreviations

All abbreviations and technical terms used in this document are defined in the List of Abbreviations section of Document 1 (Abstract, Abbreviations, and References). Key definitions directly relevant to this SRS are repeated below for convenience:

Firestore Authentication: The Google Firestore service used to manage user account creation, email verification, OTP-based MFA, session token issuance, and session lifecycle management.

Cloud Firestore: The Google Firestore NoSQL document database used as the primary structured data store for the platform. Security rules are enforced server-side on every read and write operation.

Cloudinary: The cloud-based media management platform used for secure storage and access-controlled delivery of profile images, credential documents, and prescription PDF files.

Jitsi Meet SDK: The open-source, zero-cost real-time communication SDK used to deliver video and audio consultation sessions with adaptive bitrate and DTLS-SRTP transport security.

Firestore Security Rules: Declarative server-side access control rules applied to Cloud Firestore collections, enforcing RBAC based on the authenticated user's role and data ownership conditions.

verificationStatus: A Firestore field in the users collection that holds the value 'unverified', 'pending', 'approved', or 'rejected' for healthcare professional accounts, gating access to the consultation module.

audit_logs: A dedicated Firestore collection into which all significant system events are written as immutable records for compliance and forensic purposes.

1.4 Overview

This SRS is organized as follows. Section 2 presents the overall description of the system, including its context, user characteristics, constraints, and assumptions. Section 3 specifies all functional requirements organized by system module. Section 4 specifies non-functional requirements organized by quality category. Section 5 defines system and external interface requirements. Section 6 describes system constraints. Section 7 presents the use case summary table. Section 8 defines the data requirements through the Firestore database schema. Section 9 documents the security requirements specification. Section 10 provides acceptance criteria for each principal functional requirement.

2. OVERALL DESCRIPTION

2.1 Product Perspective

The Secured Mobile Telemedicine Platform is an independent, purpose-built mobile application. It does not replace or integrate with any existing hospital information system, radiology system, or pharmacy management platform. It interfaces with the following external services through their respective APIs:

- Firebase Authentication — for identity verification and session management.
- Google Cloud Firestore — for structured data storage and RBAC enforcement.
- Firebase Cloud Messaging — for push notification delivery.
- Cloudinary — for secure media upload, storage, and delivery.
- Jitsi Meet Server (public infrastructure or self-hosted) — for real-time video/audio communication.

All inter-service communication is conducted over HTTPS/TLS 1.3. The Jitsi Meet SDK uses DTLS-SRTP for media stream security. The system does not maintain any server-side application logic beyond Firebase Security Rules and optional Firebase Cloud Functions for notification dispatch and audit logging.

2.2 Product Functions

The system provides seven functional modules:

1. Authentication and Session Management — registration, email verification, MFA, biometric login, session expiry, and role-based dashboard routing.
2. Healthcare Professional Verification — credential upload, admin review, approval/rejection notification, and consultation access gating.
3. Appointment Management — booking, confirmation, rescheduling, cancellation, and reminder notification.

4. Consultation Module — real-time Jitsi video/audio session, bandwidth monitoring, audio-only fallback, and session record creation.
5. Electronic Medical Records — clinical record creation, patient record access, prescription upload, and Cloudinary file management.
6. Notification Module — FCM push notification delivery in English or French for all appointment, consultation, and verification events.
7. Admin Module — verification request review, user account management, activity log monitoring, and system usage reporting.

2.3 User Classes and Characteristics

Three user classes interact with the system:

Patient / General Public: Primary end users of the platform. Expected to have basic smartphone literacy: ability to install apps, navigate menus, and type messages. Age range 18–50 years. May operate in English or French. May be in urban or rural settings with variable mobile connectivity. No technical background assumed.

Verified Healthcare Professional (Doctor): Registered medical practitioners or certified healthcare workers whose credentials have been administratively verified. Expected to be smartphone literate with some familiarity with digital health tools. May operate under time pressure during consultations; interface must minimize friction. Operates in English or French.

Administrator: Platform operator responsible for credential verification review, user account management, and system monitoring. Expected to have competent smartphone or tablet literacy. Accesses administrative functions through the admin dashboard. Requires no programming knowledge.

2.4 Operating Environment

The primary operating environment is Android smartphones running Android 8.0 (API level 26) and above. Secondary compatibility is provided for iOS 13.0 and above through Flutter's

cross-platform compilation. The system must remain functional under the following network conditions:

- High bandwidth (Wi-Fi or 4G): full video and audio consultation mode.
- Medium bandwidth (3G, 1–3 Mbps): video consultation with Jitsi adaptive bitrate quality reduction.
- Low bandwidth (3G degraded, 300 Kbps–1 Mbps): video consultation at minimum quality or audio-only mode.
- Very low bandwidth (below 300 Kbps): audio-only consultation mode with user prompt.

2.5 Design and Implementation Constraints

The following constraints govern the design and implementation of the system:

- The system must be implemented using the Flutter framework (Dart language) to satisfy cross-platform deployment requirements.
- The backend infrastructure must be implemented entirely using Firebase services (Authentication, Firestore, Cloud Messaging) to leverage free-tier availability and native Flutter SDK integration.
- Media storage must use Cloudinary, not Firebase Cloud Storage, for enhanced media optimization and access control URL management.
- Real-time consultation must use the Jitsi Meet SDK to satisfy the zero-cost licensing requirement for academic deployment.
- All security mechanisms must be implementable without custom cryptographic libraries, relying on industry-standard controls provided by Firebase, Cloudinary, and Jitsi.
- The system must operate within the Firebase Spark (free) plan resource limits for academic prototype deployment, targeting under 10,000 active users.

2.6 Assumptions and Dependencies

The following assumptions underpin the system requirements specified in this document:

- All users possess a smartphone with a functioning camera, microphone, and speaker for video/audio consultation.
- Users have access to a mobile data or Wi-Fi connection. Minimum required: 300 Kbps for audio-only consultation.
- Firebase services (Authentication, Firestore, FCM) maintain the uptime guarantees specified in their respective Service Level Agreements.
- Cloudinary services maintain standard API availability for upload and delivery operations.
- Jitsi public infrastructure is available for consultation sessions; self-hosted deployment is an optional enhancement outside this project's scope.
- The administrative review of healthcare professional credential documents is performed by a designated system administrator with appropriate institutional authority.

2.7 UI/UX Design: Branding, Color, and Logo

The visual identity of TeleCare was deliberately designed to reinforce the platform's core values of trust, health, and accessibility before a single screen of content is read.

Green was selected as the primary brand color. In healthcare branding, green is widely associated with health, growth, and safety, the same associations that underpin a patient's decision to trust a platform with sensitive medical information. Unlike red, which can signal alarm or urgency, or blue, which is heavily used by global health brands and can feel clinical and impersonal, green offers a calmer, more reassuring tone appropriate for a first-time user who may already be anxious about a health concern. Green also carries strong associations with growth and renewal, reflecting the platform's goal of expanding access to care rather than simply digitizing an existing, limited system.

The TeleCare logo combines a stylized cross motif, a universal symbol of medical care, with a connecting signal or pulse line, representing the real-time link the platform establishes between patient and healthcare professional regardless of physical distance. This dual symbolism was chosen to communicate, at a glance, what the platform does: deliver trustworthy medical care over a connected, real-time channel. The logo is rendered in the primary green against a white background throughout the application to maintain visual consistency and reinforce brand recognition across every screen, from the login page to the consultation interface.



3. FUNCTIONAL REQUIREMENTS

Table 3.1 specifies all functional requirements for the Secured Mobile Telemedicine Platform. Each requirement is assigned a unique identifier, a descriptive name, the primary actor(s) involved, a precise behavioral description, and a priority rating.

Table 3.1: Functional Requirements Specification

ID	Name	Actor(s)	Description	Priority
FR-01	User Registration	Patient, Doctor	User provides name, email, password, role, and preferred language. Firebase Authentication creates account and dispatches email verification link. Firestore user document created on verification.	Very High
FR-02	Multi-Factor Authentication (MFA)	All Users	On login, user enters email/password. If MFA enabled, OTP sent to registered phone via Firebase Authentication. User validates OTP before session is established.	Very High
FR-03	Biometric Authentication	All Users	On compatible devices, device-level fingerprint or face authentication offered via Flutter local_auth package as alternative to OTP.	High
FR-04	Role-Based Session Routing	All Users	Upon successful authentication, user role retrieved from Firestore users collection. Patient routed to patient dashboard; Doctor to doctor dashboard; Admin to admin dashboard.	Very High
FR-05	Healthcare Professional Credential Upload	Doctor	Doctor uploads license document and government-issued ID via Cloudinary signed upload API. Verification request document created in Firestore with status = pending.	Very High
FR-06	Admin Verification Review	Admin	Admin reviews uploaded credentials via admin dashboard. Admin approves or rejects request with optional notes. Doctor's verificationStatus updated in Firestore. Doctor notified via FCM.	Very High

FR-07	Appointment Booking	Patient	Patient selects specialty, views verified doctors, selects available slot. System checks slot availability in appointments collection. Appointment document created. Doctor notified via FCM.	High
FR-08	Appointment Confirmation	Doctor	Doctor confirms or declines pending appointment from dashboard. Patient receives FCM confirmation or cancellation notification. 24-hour reminder scheduled on confirmation.	High
FR-09	Appointment Rescheduling and Cancellation	Patient, Doctor	Either party may reschedule or cancel a confirmed appointment. Appointment status updated in Firestore. Counterparty notified via FCM. Audit log entry created.	Medium
FR-10	Real-Time Video Consultation	Patient, Verified Doctor	At scheduled time, FCM delivers consultation-start notification with Jitsi room ID. Both parties initialize Jitsi Meet SDK session. Adaptive bitrate adjusts video quality to network conditions.	Very High
FR-11	Audio-Only Fallback Mode	Patient, Verified Doctor	If network bandwidth below threshold, system prompts user to switch to audio-only. User can confirm switch. Jitsi session continues in audio-only mode. Mode recorded in consultations document.	High
FR-12	Secure Messaging	Patient, Verified Doctor	Asynchronous text messages exchanged within appointment context via messages Firestore collection. Messages restricted by Firebase Security Rules to sender, receiver, and admin.	Medium
FR-13	Medical Record Creation	Verified Doctor	After consultation, doctor creates structured EMR document in medical_records collection: diagnosis, symptoms, treatment plan. Firebase Security Rules enforce role-based write access.	Very High
FR-14	Prescription Upload	Verified Doctor	Doctor uploads prescription PDF via Cloudinary signed upload API.	High

			Returned URL stored in medical_records document. Patient notified of availability via FCM.	
FR-15	Patient Medical Record Access	Patient	Patient views own medical records and prescriptions from patient dashboard. Firebase Security Rules restrict access to owning patient, attending doctor, and admin.	High
FR-16	Push Notifications	All Users	FCM delivers: appointment confirmations, 24-hour reminders, consultation-start alerts, prescription availability notices, and verification status updates. Content delivered in user's selected language.	High
FR-17	Bilingual Interface	All Users	All UI text, error messages, and notifications available in English and French. Language selected at registration; changeable in profile settings at any time via Flutter intl framework.	High
FR-18	Activity Logging	Admin	All significant events (login, record access, prescription upload, admin action, consultation start/end) written to audit_logs Firestore collection with userId, action, resourceId, and timestamp.	Very High
FR-19	Admin User Management	Admin	Admin can view, deactivate, and reactivate user accounts from admin dashboard. Changes recorded in audit_logs.	High
FR-20	Admin Dashboard	Admin	Admin accesses pending verification requests, active users, recent activity logs, and system usage statistics from dedicated admin interface.	High

4. NON-FUNCTIONAL REQUIREMENTS

Table 4.1 specifies the non-functional requirements governing the quality attributes and operational constraints of the system. Requirements are organized by category and assigned unique identifiers for traceability.

Table 4.1: Non-Functional Requirements Specification

ID	Category	Requirement	Priority
SEC-01	Authentication	All user sessions must be secured through Firebase Authentication. MFA must be enforced for doctor and admin accounts. Session tokens expire after 30 minutes of inactivity.	Very High
SEC-02	Data Encryption at Rest	All Firestore documents and Cloudinary files must be encrypted at rest using AES-256, enforced by Google Cloud and Cloudinary infrastructure respectively.	Very High
SEC-03	Data Encryption in Transit	All API calls, Firestore connections, FCM payloads, and Cloudinary upload/download requests must be transmitted over HTTPS/TLS 1.3.	Very High
SEC-04	Access Control	Firebase Security Rules must enforce RBAC on every Firestore read and write operation. Patients may only access their own records. Doctors access records of patients they have treated. Admins have full read access.	Very High
SEC-05	Healthcare Professional Gating	Doctor and healthcare worker accounts must have verificationStatus = approved before accessing appointment booking, video consultation, EMR creation, or prescription upload modules.	Very High
SEC-06	Audit Trail	All significant system interactions must be written to the audit_logs collection. Audit log entries must be immutable after creation. Logs must be accessible to admin accounts only.	High
SEC-07	Secure Media Upload	All file uploads must use Cloudinary signed upload APIs. File type must be restricted to PDF and image formats. File size must not exceed 5	High

		MB per upload. Access-controlled delivery URLs must be used for all retrieval.	
PERF-01	Screen Load Time	Core application screens (dashboard, appointment list, consultation screen) must load within 3 seconds on a 3G mobile connection (minimum 1 Mbps downlink).	High
PERF-02	Video Consultation Start Time	Video consultation session must initialize within 5 seconds of both parties tapping 'Join Consultation' under normal network conditions.	High
PERF-03	Low-Bandwidth Operation	Video consultation must remain functional at connection speeds as low as 300 Kbps through Jitsi's adaptive bitrate control. Audio-only fallback must activate below this threshold if user consents.	Very High
PERF-04	Notification Latency	FCM push notifications for appointment confirmations and consultation-start alerts must be delivered within 5 seconds of trigger under normal network conditions.	Medium
REL-01	System Availability	Platform uptime must exceed 99% per calendar month, excluding planned maintenance windows announced 24 hours in advance.	High
REL-02	Data Integrity	All Firestore write operations must be atomic. Failed writes must not result in partial data persistence. Cloudinary uploads must be confirmed before URLs are written to Firestore.	Very High
REL-03	Error Handling	All API errors and network failures must be caught and presented to the user as localized error messages in the user's selected language. No raw error codes or stack traces must be exposed to end users.	High
SCAL-01	User Scalability	The Firestore database architecture must support a minimum of 10,000 registered users and 1,000 concurrent sessions without schema redesign, leveraging Firestore's horizontal scaling.	Medium
SCAL-02	Storage Scalability	Cloudinary media storage must support growth without reconfiguration. Total stored data capacity must accommodate at least 100,000 prescription and document uploads.	Medium

USE-01	Usability	The application must be navigable by users with basic smartphone literacy (defined as ability to install apps, navigate menus, and type text messages) in both English and French.	High
USE-02	Accessibility	Font sizes must be no smaller than 12pt equivalent on standard mobile screen sizes. Interactive elements must have minimum touch target sizes of 48x48 dp in accordance with Android and iOS guidelines.	Medium
MAIN-01	Modularity	The application must be structured in seven independent modules as specified in the system design. Each module must be independently testable without deploying the full application.	High
MAIN-02	Code Documentation	All Dart source files must include inline documentation comments for all public functions, classes, and Firebase Security Rule definitions.	Medium
COMP-01	HIPAA Alignment	The system design must align with HIPAA Security Rule requirements for access control, audit controls, integrity, and transmission security, as applicable to the Cameroonian academic context.	High
COMP-02	ISO/IEC 27001 Alignment	The security mechanism design must address the information security controls specified in ISO/IEC 27001:2013, including access control, cryptography, operations security, and incident management.	High

5. SYSTEM INTERFACE REQUIREMENTS

5.1 User Interface Requirements

Table 5.1 describes the principal screens and their interface requirements.

Table 5.1: User Interface Requirements

Screen	Actor	Interface Requirements
Patient Dashboard	Patient	Home screen after login showing upcoming appointments, recent notifications, and quick-access buttons for booking, medical records, and messaging.
Doctor Dashboard	Verified Doctor	Home screen after verified login showing appointment calendar, pending requests, recent consultations, and quick access to EMR creation.
Admin Dashboard	Admin	Management screen showing pending verification requests count, recent activity log entries, user account statistics, and navigation to management functions.
Registration Screen	All Users	Multi-step form: personal details, role selection (patient / doctor / admin), language preference (English / French), email and password entry, and terms acknowledgement.
Credential Upload Screen	Doctor	Presented immediately after doctor registration. Two upload fields: professional license document and government-issued ID. File type and size validation feedback. Progress indicator.
Appointment Booking Screen	Patient	Specialty selection grid, verified doctor list filtered by specialty, doctor profile view, date picker, available time slot selector, and booking confirmation button.
Consultation Screen	Patient, Doctor	Full-screen Jitsi Meet SDK view with video/audio controls, microphone mute, camera toggle, chat overlay, bandwidth indicator, audio-only switch prompt, and end-consultation button.
Medical Record Form	Verified Doctor	Structured form: diagnosis text field, symptoms multi-line field, treatment plan field, prescription file upload, optional attachment upload, and save button.

Medical Records View	Patient	Chronological list of consultation records. Each record expandable to show diagnosis, treatment plan, and prescription download link. Search by date filter.
Settings Screen	All Users	Language toggle (English / French), notification preferences, password change, account deactivation, and profile image update via Cloudinary upload.

General UI requirements applicable to all screens: minimum font size 12pt equivalent; interactive elements minimum 48x48 dp touch target; all text and labels available in English and French; loading indicators displayed for all network operations exceeding 500ms; error messages displayed in user's selected language; back navigation available on all non-root screens.

5.2 Hardware Interfaces

The system interfaces with the following hardware components of the user's device:

- Camera: used for video consultation (front-facing preferred; rear as fallback) and profile image capture. Camera permission must be requested at runtime.
- Microphone: used for audio consultation and voice communication. Microphone permission must be requested at runtime.
- Speaker / Earpiece: used for audio output during consultations. System must switch audio output appropriately between earpiece, speaker, and connected Bluetooth audio devices.
- Fingerprint sensor / Face recognition system: used for biometric authentication via Flutter local_auth package. System gracefully degrades to OTP-based MFA on devices without biometric hardware.
- Network interface: 3G/4G/5G cellular modem or Wi-Fi adapter for all network communication.

5.3 Software Interfaces

The system interfaces with the following external software services:

Firestore Authentication SDK (Flutter): Provides user registration, email verification, OTP, biometric token management, and session token issuance and validation. Interface: Firestore Auth Flutter plugin (firebase_auth).

Firestore SDK (Flutter): Provides real-time read/write access to the NoSQL document database with offline caching and server-side security rule enforcement. Interface: cloud_firestore Flutter plugin.

Firestore Cloud Messaging SDK (Flutter): Provides push notification receipt and foreground/background notification handling. Interface: firebase_messaging Flutter plugin.

Cloudinary Upload API (HTTPS): Provides signed upload endpoints for media file upload with file type and size validation. Response returns access-controlled delivery URL. Interface: HTTPS POST to Cloudinary upload endpoint.

Jitsi Meet SDK (Flutter): Provides real-time video and audio communication with adaptive bitrate, DTLS-SRTP security, and room management. Interface: jitsi_meet_flutter_sdk plugin.

Flutter local_auth plugin: Provides device-level biometric authentication (fingerprint, face recognition) as alternative to OTP-based MFA.

5.4 Communication Interfaces

All communications between the mobile application and external services use the following protocols:

- Firestore Authentication, Firestore, FCM: HTTPS/TLS 1.3 via the Firestore Flutter SDK.
- Cloudinary upload and delivery: HTTPS/TLS 1.3 to Cloudinary API endpoints.
- Jitsi Meet SDK: DTLS-SRTP for media streams; WebSocket over TLS 1.3 for signaling.

6. SYSTEM CONSTRAINTS

The following constraints limit the design and implementation choices available for this system:

SC-01 — Academic Budget Constraint: The system must operate within the free-tier resource limits of Firebase Spark plan and Cloudinary free plan. No paid cloud services may be used in the prototype implementation.

SC-02 — Single Developer Constraint: The system is implemented by a single developer within the time constraints of a final year undergraduate project (2024/2025 academic year). Complexity must be scoped to ensure full module implementation within this constraint.

SC-03 — No Custom Cryptography: All cryptographic operations must be delegated to industry-standard libraries and services (Firebase, Cloudinary TLS, Jitsi DTLS-SRTP). Custom cryptographic algorithms must not be implemented.

SC-04 — No AI/ML Integration: The system must not incorporate machine learning models, AI diagnostic tools, or automated clinical decision support systems in this version.

SC-05 — No Payment Processing: The system must not implement any payment gateway, billing system, or financial transaction processing.

SC-06 — Internet Connectivity Required: Real-time consultation requires active internet connectivity. The system does not support fully offline operation. Firestore offline caching provides read-only access to cached records when connectivity is unavailable.

SC-07 — Admin Availability Dependency: Healthcare professional account activation depends on manual administrative review. System design must communicate clearly to awaiting doctors that verification is subject to admin availability.

Use Case Descriptions

The following tables provide the complete structured description for each principal use case. These descriptions should be inserted into Chapter Three, Section 6.1, immediately after Figure 3.7 (Use Case Diagram). Each table follows the standard UML use case description format.

UC-01: Register / Login with MFA

Use Case UC-01: Register / Login with Multi-Factor Authentication	
Use Case ID	UC-01
Use Case Name	Register / Login with Multi-Factor Authentication
Actor(s)	Patient, Doctor (unverified), Admin
Trigger	User opens the application for the first time (registration) or returns to log in (login)
Pre-conditions	For registration: user has not previously created an account. For login: user has a registered and email-verified account.
Main Flow	1. User opens the application and selects 'Register' or 'Log In'. 2. For registration: user provides name, email, password, role, and preferred language (English or French). 3. Firebase Authentication creates the account and dispatches an email verification link. 4. User clicks the verification link in their email; the application confirms email verification. 5. Application creates the user profile document in the Firestore users collection with the declared role. 6. For login: user enters email and password; Firebase Authentication validates credentials. 7. If MFA is enabled, Firebase Authentication sends an OTP to the user's registered phone number. 8. User enters the OTP; the application validates it. 9. On compatible devices, biometric authentication (fingerprint or facial recognition) is offered as an alternative to OTP entry via the Flutter local_auth package. 10. Upon successful authentication, the application retrieves the user's role from Firestore and routes the user to their role-appropriate dashboard. 11. A LOGIN event is written to the audit_logs Firestore collection.
Alternative Flow	Step 7 (No MFA): If MFA is not enabled, OTP step is skipped and authentication proceeds directly to biometric check or dashboard routing. Step 9 (Biometric unavailable): If the device does not support biometrics, only OTP-based MFA is offered. Step 3 (Doctor role): Doctor registrants are redirected to the credential upload screen (UC-02) immediately after email verification.
Post-conditions	User is authenticated with a valid Firebase session token. Role is stored in the application state. User is routed to their role-appropriate dashboard. Login event is recorded in audit_logs.

Exceptions	Invalid credentials: application displays an error message and increments failed attempt counter. After 5 failed attempts, the account is locked for 30 minutes. Expired OTP: application prompts user to request a new OTP. Email not verified: application displays a 'Please verify your email' message and offers to resend the verification link.
------------	--

UC-02: Healthcare Professional Credential Verification

Use Case UC-02: Healthcare Professional Credential Verification	
Use Case ID	UC-02
Use Case Name	Healthcare Professional Credential Verification
Actor(s)	Doctor (primary), Admin (reviewer)
Trigger	Doctor completes registration (UC-01) and is redirected to the credential upload screen
Pre-conditions	Doctor has a registered and email-verified account with role = doctor and verificationStatus = unverified.
Main Flow	1. Doctor is presented with the credential upload screen after completing registration. 2. Doctor selects and uploads a professional registration or license document (PDF or image, maximum 5 MB) via the Cloudinary signed upload API. 3. Doctor selects and uploads a government-issued identification document via the Cloudinary signed upload API. 4. Application creates a verification_requests document in Firestore with status = pending, storing the Cloudinary document URLs. 5. Application updates the doctor's users document: verificationStatus = pending. 6. A push notification is dispatched to the Admin via Firebase Cloud Messaging: 'New verification request from [doctorName]'. 7. Admin opens the admin dashboard and navigates to pending verification requests. 8. Admin reviews the uploaded credential and identification documents. 9. Admin selects Approve or Reject with optional review notes. 10. If approved: verification_requests.status and users.verificationStatus are updated to 'approved'; a push notification is dispatched to the Doctor. 11. If rejected: verification_requests.status and users.verificationStatus are updated to 'rejected'; a push notification including review notes is dispatched to the Doctor. 12. A verification event is written to the audit_logs collection.

Alternative Flow	Step 2 (Invalid file type): If the uploaded file type is not PDF or image, the application displays a validation error and prompts the doctor to reselect a valid file. Step 2 (File too large): If the file exceeds 5 MB, the application displays an error and prompts re-upload. Step 10 (Rejection with resubmission): Rejected doctors are permitted to resubmit corrected credentials; a new verification_requests document is created and the workflow repeats from Step 4.
Post-conditions	Doctor's verificationStatus is updated to 'approved' or 'rejected' in Firestore. Doctor is notified of the outcome. If approved, the consultation module is accessible to the Doctor. Verification event is recorded in audit_logs.
Exceptions	Cloudinary upload failure: application displays an upload error and prompts retry. Admin unavailable: verification remains pending until an admin reviews the request. Fraudulent documents: Admin rejects with review notes; case may be escalated through institutional channels outside the system.

UC-03: Book Appointment

Use Case UC-03: Book Appointment	
Use Case ID	UC-03
Use Case Name	Book Appointment
Actor(s)	Patient
Trigger	Patient selects 'Book Appointment' from the patient dashboard
Pre-conditions	Patient is authenticated with a valid session. At least one verified doctor is registered on the platform.
Main Flow	1. Patient navigates to the 'Find a Doctor' screen and selects a medical specialty. 2. Application retrieves the list of verified doctors in the selected specialty from Firestore (verificationStatus = approved). 3. Patient selects a preferred doctor and views their available time slots. 4. Application queries the appointments Firestore collection for existing confirmed bookings for the selected doctor on the requested date. 5. Patient selects an available date and time slot and taps 'Book'. 6. Application creates an appointment document in the appointments collection: {patientId, doctorId, scheduledDate, scheduledTime, status: 'pending', createdAt}. 7. A push notification is dispatched to the Doctor via Firebase Cloud Messaging: 'New appointment request from [patientName]'.

	8. Doctor confirms or declines the appointment from their dashboard. 9. If confirmed: appointments.status is updated to 'confirmed'; a confirmation notification is dispatched to the Patient. 10. A reminder notification is scheduled for 24 hours before the appointment. 11. Appointment booking event is written to audit_logs.
Alternative Flow	Step 5 (Slot unavailable): If the selected time slot has been booked by another patient before the booking is confirmed, the application notifies the Patient and prompts them to select an alternative slot. Step 8 (Doctor declines): If the Doctor declines, appointments.status is updated to 'cancelled' and the Patient receives a cancellation notification.
Post-conditions	Appointment document is created in Firestore with status 'confirmed'. Patient and Doctor have received confirmation notifications. Reminder notification is scheduled. Booking event is recorded in audit_logs.
Exceptions	No available doctors: Application displays a message indicating no verified doctors are currently available in the selected specialty. Network failure: Application displays a connectivity error; the booking attempt is not persisted.

UC-04: Join / Conduct Remote Consultation

Use Case UC-04: Join / Conduct Remote Consultation	
Use Case ID	UC-04
Use Case Name	Join / Conduct Remote Consultation
Actor(s)	Patient, Verified Doctor
Trigger	Scheduled consultation time is reached; both Patient and Doctor receive consultation-start push notifications
Pre-conditions	Appointment status = 'confirmed'. Both Patient and Doctor have active authenticated sessions. Doctor's verificationStatus = 'approved'.
Main Flow	1. Firebase Cloud Messaging dispatches consultation-start notifications to both Patient and Doctor containing the Jitsi room identifier. 2. Patient taps 'Join Consultation' in the patient dashboard. 3. Application verifies the Firebase session token; prompts re-authentication if the token has expired. 4. Application retrieves the Jitsi room identifier from the consultations Firestore document. 5. Application initializes the Jitsi Meet SDK with the room identifier and the Patient's display identity. 6. Application measures available network bandwidth via Jitsi quality indicators.

	7. If bandwidth is sufficient: session initializes in video-and-audio mode. 8. If bandwidth is below threshold: application displays a prompt offering audio-only mode; Patient selects preferred mode. 9. Doctor follows identical steps 2 to 8 on their device. 10. Jitsi establishes the secure communication session between both participants using DTLS-SRTP. 11. Adaptive bitrate control dynamically adjusts video quality throughout the session in response to real-time network conditions. 12. Upon session termination, application updates the consultations document: {endTime, mode, status: 'completed'}. 13. Consultation-end event is written to audit_logs. 14. Doctor is prompted to create a medical record for the consultation (UC-05).
Alternative Flow	Step 8 (Very low bandwidth; automatic switch): If bandwidth drops to a critically low level during the session, the Jitsi SDK automatically reduces video quality; if quality is unacceptable, the application offers immediate switch to audio-only mode. Step 9 (Doctor absent): If the Doctor does not join within a configurable grace period (e.g., 10 minutes), the Patient receives a notification and the appointment is flagged for rescheduling.
Post-conditions	Consultation session is completed and recorded in Firestore with end time, mode, and status. Consultation-end event is in audit_logs. Doctor is directed to create the medical record.
Exceptions	Jitsi connection failure: Application displays a connection error and offers retry. Both parties are notified. Session data for the attempted consultation is logged in audit_logs with outcome = 'failed'. Authentication failure: If either party's session token has expired and re-authentication fails, the consultation cannot be initiated.

UC-05: Create Medical Record and Upload Prescription

Use Case UC-05: Create Medical Record and Upload Prescription	
Use Case ID	UC-05
Use Case Name	Create Medical Record and Upload Prescription
Actor(s)	Verified Doctor
Trigger	Doctor taps 'Create Record' after a consultation is completed (UC-04)
Pre-conditions	Consultation status = 'completed'. Doctor is authenticated with verificationStatus = 'approved'.

Main Flow	1. Doctor is presented with the medical record creation form. 2. Doctor enters the consultation diagnosis, presenting symptoms, and proposed treatment plan. 3. Doctor optionally uploads a prescription document (PDF) via the Cloudinary signed upload API. 4. Cloudinary returns a secure access-controlled URL for the uploaded prescription file. 5. Application creates a medical_records document in Firestore: {patientId, doctorId, consultationId, diagnosis, symptoms, treatmentPlan, prescriptionUrl, createdAt}. 6. Firebase Security Rules verify that the authenticated user is the attending Doctor for the consultation before the write is permitted. 7. A push notification is dispatched to the Patient: 'Your medical record and prescription are available'. 8. Record creation event is written to audit_logs.
Alternative Flow	Step 3 (No prescription): If no prescription is required, the prescriptionUrl field is left null and the record is saved without a file attachment. Step 3 (Invalid file): If the uploaded file is not a PDF or exceeds the maximum size, the application displays a validation error and prompts re-upload.
Post-conditions	Medical record document is created in Firestore and is accessible to the Patient and the attending Doctor under Firebase Security Rules. Patient has received a notification. Record creation event is in audit_logs.
Exceptions	Cloudinary upload failure: Application displays an upload error; Doctor may retry. The record can be saved without the prescription and updated later. Firestore write failure: Application displays an error and prompts retry; record is not partially persisted.

UC-06: Review Verification Request (Admin)

Use Case UC-06: Review Verification Request	
Use Case ID	UC-06
Use Case Name	Review Verification Request
Actor(s)	Admin
Trigger	Admin receives a push notification of a pending verification request, or navigates to the admin dashboard
Pre-conditions	Admin is authenticated with role = admin. At least one verification_requests document exists with status = 'pending'.

Main Flow	1. Admin navigates to the 'Verification Requests' section of the admin dashboard. 2. Application retrieves all pending verification_requests documents from Firestore. 3. Admin selects a pending request and views the Doctor's submitted license document and identification document via the Cloudinary access-controlled URLs. 4. Admin evaluates the submitted documents against credential validity criteria. 5. Admin selects 'Approve' or 'Reject' and optionally enters review notes. 6. If Approve: application updates verification_requests.status = 'approved' and users.verificatioStatus = 'approved'. 7. If Reject: application updates verification_requests.status = 'rejected' and users.verificatioStatus = 'rejected'; review notes are stored. 8. Push notification is dispatched to the Doctor informing them of the decision. 9. Verification review event is written to audit_logs.
Alternative Flow	Step 4 (Insufficient documentation): Admin may request additional documentation by rejecting with review notes specifying what is missing, enabling the Doctor to resubmit.
Post-conditions	Verification request status is updated in Firestore. Doctor is notified. If approved, Doctor's account is activated for consultation services. Review event is in audit_logs.
Exceptions	Document inaccessible (Cloudinary link expired): Admin contacts the Doctor to resubmit credentials via the platform. Firestore write failure: Application displays an error; Admin retries.

7. USE CASE SUMMARY

Table 7.1 provides a complete summary of all identified use cases for the system, mapping each to its primary actor, trigger condition, and priority. Full use case description tables (with main flow, alternative flow, pre-conditions, post-conditions, and exceptions) are provided in the UML modeling document.

Table 7.1: Use Case Summary

ID	Use Case Name	Primary Actor(s)	Trigger	Priority
UC-01	Register / Login with MFA	Patient, Doctor, Admin	User opens the app to register or log in	Very High
UC-02	Healthcare Professional Credential Verification	Doctor (primary), Admin (reviewer)	Doctor completes registration	Very High
UC-03	Book Appointment	Patient	Patient selects 'Book Appointment' from dashboard	High
UC-04	Confirm / Decline Appointment	Verified Doctor	Doctor receives FCM appointment request notification	High
UC-05	Join / Conduct Remote Consultation	Patient, Verified Doctor	Scheduled consultation time is reached	Very High
UC-06	Switch to Audio-Only Mode	Patient, Verified Doctor	Bandwidth drops below acceptable video threshold during consultation	High
UC-07	Send Secure Message	Patient, Verified Doctor	User navigates to messaging screen within appointment context	Medium
UC-08	Create Medical Record	Verified Doctor	Consultation session is terminated	Very High
UC-09	Upload Prescription	Verified Doctor	Doctor taps 'Upload Prescription' in medical record form	High

UC-10	View Medical Records	Patient	Patient navigates to 'My Records' in patient dashboard	High
UC-11	Review Verification Request	Admin	Admin receives FCM notification of pending verification request	Very High
UC-12	Manage User Accounts	Admin	Admin navigates to 'User Management' in admin dashboard	High
UC-13	Monitor Activity Logs	Admin	Admin navigates to 'Activity Logs' in admin dashboard	High

8. DATA REQUIREMENTS

8.1 Database Overview

The system uses Google Cloud Firestore as its primary NoSQL document database. Data is organized into eight top-level collections. All documents are assigned a UUID as their document identifier. All collections are governed by Firebase Security Rules enforcing RBAC based on the requesting user's authentication state, role, and data ownership conditions.

8.2 Data Retention and Privacy

Patient medical records must be retained indefinitely to support continuity of care. User account data must be retained for the lifetime of the account. Audit log entries must be immutable after creation and retained for a minimum of 12 months. Upon user account deactivation, personal identifiers in non-medical records may be anonymized. Prescription documents stored in Cloudinary must be accessible via access-controlled delivery URLs only; public URL access must be disabled.

8.3 Data Validation Rules

The following validation rules must be enforced at the application layer and, where applicable, through Firebase Security Rules:

- Email addresses must conform to RFC 5322 format; uniqueness enforced by Firebase Authentication.
- Passwords must be a minimum of 8 characters; Firebase Authentication enforces this constraint.
- Uploaded credential documents must be PDF or image format (JPEG, PNG) and must not exceed 5 MB.
- Appointment date and time must be in the future at the point of booking.

- Medical record fields (diagnosis, symptoms, treatment plan) must be non-empty strings with a maximum of 2,000 characters each.
- Notification content must not exceed 200 characters to comply with FCM payload size limits.

9. SECURITY REQUIREMENTS SPECIFICATION

This section consolidates the security requirements of the system into a formal specification organized by security domain. These requirements supplement the security non-functional requirements in Section 4 and the security mechanism inventory in Chapter Three.

9.1 Identification and Authentication

- SR-IA-01: All users must be uniquely identified by a Firebase Authentication UID before any system resource is accessible.
- SR-IA-02: Passwords must be stored as cryptographic hashes managed exclusively by Firebase Authentication. Plaintext passwords must never be stored or logged.
- SR-IA-03: Multi-factor authentication must be enforced for all doctor and admin accounts. Patient accounts must offer MFA as an opt-in feature.
- SR-IA-04: OTP codes must expire after 5 minutes. Expired OTPs must be rejected and a new OTP dispatched only upon user request.
- SR-IA-05: After 5 consecutive failed login attempts, the account must be locked for 30 minutes. The user must be informed of the lockout duration.
- SR-IA-06: Biometric authentication, where available, must use the device's secure biometric API (Android BiometricPrompt or iOS LocalAuthentication). Biometric data must never leave the device.

9.2 Access Control

- SR-AC-01: All Firestore operations must be evaluated against Firebase Security Rules before execution. Unauthenticated requests must be rejected for all collections.
- SR-AC-02: Patient records (medical_records, appointments, messages, and notifications) must be readable only by the owning patient, the attending doctor, and admin accounts.

- SR-AC-03: Medical record write operations must be restricted to the attending doctor and admin accounts.
- SR-AC-04: The `verification_requests` collection must be readable only by the submitting doctor and admin accounts. Write access for approval/rejection is restricted to admin accounts.
- SR-AC-05: The `audit_logs` collection must be write-accessible only through server-side Firebase Cloud Functions. Direct client-side writes to `audit_logs` must be blocked by Security Rules.
- SR-AC-06: The admin dashboard and all admin-only endpoints must be accessible only to users with `role = admin` in the Firestore `users` collection.

9.3 Data Protection

- SR-DP-01: All data transmitted between the mobile application and Firebase services must be encrypted using TLS 1.3. Downgrade to TLS 1.2 or earlier must not be permitted.
- SR-DP-02: All Firestore documents must be encrypted at rest using AES-256 as provided by Google Cloud infrastructure.
- SR-DP-03: All files stored in Cloudinary must be encrypted at rest. Access-controlled delivery URLs must be used for all prescription and credential document access.
- SR-DP-04: All Jitsi Meet SDK media streams must be secured by DTLS-SRTP. Unencrypted media transmission must not be permitted.

9.4 Audit and Accountability

- SR-AA-01: The `audit_logs` Firestore collection must record the following events as a minimum: user login, user logout, failed login attempt, doctor credential submission, admin verification decision, appointment creation, appointment cancellation, consultation start, consultation end, medical record creation, prescription upload, and admin user account modification.

- SR-AA-02: Each audit log entry must contain: logId (UUID), userId, action (string constant), resourceType, resourceId, timestamp (server-generated), and outcome (success/failure).
- SR-AA-03: Audit log entries must be immutable. Firebase Security Rules must block all client-side update and delete operations on the audit_logs collection.

9.5 Healthcare Professional Verification Security

- SR-HV-01: A user with role = doctor or healthcare_worker must have verificationStatus = approved in the Firestore users' collection before any of the following features are accessible: appointment booking as a provider, video consultation initiation, medical record creation, and prescription upload.
- SR-HV-02: The verificationStatus field must only be updatable by admin accounts or server-side Firebase Cloud Functions. Client-side updates to verificationStatus by doctor accounts must be blocked by Firebase Security Rules.
- SR-HV-03: Uploaded credential documents must be stored in Cloudinary with access-controlled delivery URLs. Direct public access to credential documents must be disabled.

10. ACCEPTANCE CRITERIA

The following acceptance criteria define the conditions that must be satisfied for the implemented system to be considered as fulfilling the requirements specified in this document. Each criterion corresponds to a principal functional or non-functional requirement.

Table 10.1: Acceptance Criteria

Ref	Requirement	Acceptance Criterion
FR-01	User Registration	A new user can complete registration, receive and click the email verification link, and have their profile document created in Firestore. The registration form validates input and displays appropriate error messages in the user's selected language.
FR-02	MFA Login	A registered doctor account cannot complete login without providing a valid OTP. Expired OTPs are rejected. The system locks the account after 5 consecutive failed attempts.
FR-05	Credential Upload	A doctor registrant can upload a license document and ID document. Both files are stored in Cloudinary and a verification_requests document is created in Firestore with status = pending.
FR-06	Admin Verification	An admin can view pending verification requests, open uploaded documents via Cloudinary URL, and approve or reject the request. The doctor's verificationStatus is updated and the doctor receives a push notification.
FR-07/08	Appointment Booking	A patient can book an appointment with a verified doctor. The doctor receives a push notification and can confirm or decline. The patient receives a corresponding notification. A 24-hour reminder is dispatched.
FR-10	Video Consultation	Both patient and verified doctor receive FCM consultation-start notifications and can join a Jitsi session. Video and audio are functional. The consultation end time is written to Firestore.
FR-11	Audio-Only Fallback	Under simulated low-bandwidth conditions, the system displays an audio-only prompt. Confirming the prompt disables the video track and the consultation continues in audio-only mode.

FR-13/14	Medical Record	After consultation, a verified doctor can create a medical record with diagnosis, symptoms, and treatment plan, and upload a prescription PDF. The prescriptionUrl is stored in the medical_records document.
FR-15	Patient Record Access	A patient can view their medical records and download their prescription PDF. Attempting to access another patient's records returns a permission denied error.
SEC-01	MFA Enforcement	An unverified doctor (verificationStatus != approved) attempting to access the appointment booking or consultation module receives an access restricted message without exception.
FR-17	Bilingual Interface	All application screens, error messages, and push notifications display in the user's selected language. Language can be changed in profile settings and the change is immediately reflected across the UI.
FR-18	Activity Logging	Login, consultation start/end, medical record creation, and admin verification actions each produce a corresponding entry in the audit_logs Firestore collection with correct userId, action, and timestamp fields.
PERF-03	Low-Bandwidth Operation	Under a simulated 500 Kbps network condition, the Jitsi session remains active with reduced video quality. Under a 200 Kbps condition, the audio-only prompt is displayed.
REL-01	System Availability	The Firebase backend services maintain availability consistent with the Firebase SLA during the testing period. No data is lost during simulated connection interruptions.

APPENDIX: DOCUMENT REVISION HISTORY

Version	Date	Author	Description of Change
1.0	May 2025	ABILATEZIE VIN- WILSON ANU	Initial version; complete SRS for final year project submission, Academic Year 2024/2025.